SHOPFLOW CURRICULUM · PART B · 2026 EDITION

Frontend Engineering

Phases 5–6 · Modern React, TypeScript & Production-Grade UI Engineering

HTML, CSS, Tailwind & shadcn/ui, JavaScript, and TypeScript fundamentals, followed by a modern React 19 + TypeScript application layer — component architecture, forms, data fetching, state management, accessibility, testing, and performance — wired to the real Spring Boot API you built in Part A. Every project extends one continuously evolving ShopFlow codebase; nothing is rebuilt from scratch.

Phases 5–6 | 15 Topics | 8–10 Weeks | Beginner → Intermediate | 8–12 hrs/week | A11y & Testing Built In

PHASES	TOPICS	ESTIMATED DURATION	DIFFICULTY	SUGGESTED PACE	PROJECTS	DOCUMENTATION SOURCES	CONTINUITY
5–6 of 10	15	8–10 weeks	Beginner → Intermediate	8–12 hrs/week · 2–3 topics/week (part-time default)	2–4 hands-on builds per topic, all on one live codebase	2+ official/primary sources per topic	Every project extends the ShopFlow app started in Part A

What Changed in This Revision

Reviewed against professional 2026 frontend practice. **[NEW]** = added topic/subtopic. **[UPDATED]** = deepened, reordered, or replaced. Unmarked topics carry over from the original roadmap. This edition also removes every standalone "toy" project — Todo lists, weather widgets, throwaway portfolios — in favor of building the same skill directly inside ShopFlow, so each topic leaves the product measurably better instead of resetting to a blank slate.

REPLACED Bootstrap → **Tailwind CSS & shadcn/ui**. Radix-primitive, copy-in components are the pattern dominating new React libraries, and feed directly into the Design System & Component Library work in Phase 6.

DEEPENED **CSS** adds container queries and logical properties — the real answer to component-level responsiveness, since a component rarely knows its own viewport.

DEEPENED **JavaScript** adds the event loop (microtasks vs. macrotasks) — the mechanism behind most async bugs students later hit in React.

DEEPENED **React + TypeScript** adds React 19 (Actions, useOptimistic, useActionState, use(), ref-as-prop), useReducer, custom hooks, error boundaries, and SPA focus management.

ADDED **Component Architecture, Design Systems & Composition Patterns** — compound components, controlled vs. uncontrolled, design tokens, prop drilling vs. lifting state, taught explicitly.

ADDED **Forms with React Hook Form + Zod** — hand-rolled form state is exactly the repetitive work the ecosystem already solved.

REPLACED Axios & API Integration → **Axios + TanStack Query**. Axios stays for the client/interceptor layer; caching and background refetching move to React Query.

ADDED **State Management: Context vs. Zustand** — separates server state (TanStack Query) from client/UI state, and names when each tool is actually the right one.

ADDED **Testing & Accessibility Audits: Vitest, RTL, axe-core & Playwright** — component tests, automated a11y assertions, and one E2E flow, immediately after components are built.

ADDED **Performance: Code-Splitting, Suspense & Core Web Vitals** — the same "measure before optimizing" instinct from Part A's database phase, applied to the frontend, with a scored Lighthouse CI gate.

ADDED **Next.js Primer** — SSR, file-based routing, Server Components, and structured-data SEO in one guided rebuild of a single ShopFlow page. Deliberately scoped as a primer, not a migration.

UPDATED Every phase now closes with a **Ship-It Checklist** (testing, accessibility, debugging, production-readiness) instead of leaving those as an afterthought.

Topics Covered

Phase 5 — Foundations
HTML · CSS (+ Container Queries) · Tailwind CSS & shadcn/ui · JavaScript ES6+ (+ Event Loop) · TypeScript

Phase 6 — Application Layer
React 19 + TypeScript · Component Architecture & Design Systems · Forms (RHF + Zod) · Axios + TanStack Query · State Management (Context vs. Zustand) · Types of APIs · Testing & A11y Audits (Vitest/RTL/axe/Playwright) · Performance & Core Web Vitals · Frontend Dev Tools · Next.js Primer

Next: with a working, tested, accessible, performance-checked React frontend talking to a real backend, move on to Part C — Deployment, Cloud & Integrations.

PHASE 5 OF 10

Frontend Foundations

HTML · CSS (+ Container Queries) · Tailwind & shadcn/ui · JavaScript (ES6+) · TypeScript

Before React, get fluent in the languages the browser actually runs. This is what lets you debug a layout bug or a broken event handler without guessing — and what separates someone who can use React from someone who understands it.

Continuity: every project in this phase works directly on the ShopFlow Storefront you scaffold in Topic 1 (product listing, product detail, cart) and on a companion developer portfolio site used to showcase it. No project starts a new codebase — each one refines the last commit.

1 HTML

The structural skeleton of every page — semantic, accessible markup that means something, not just divs everywhere.

Subtopics: semantic HTML5 elements · forms & validation attributes · accessibility (ARIA, alt text) · meta tags, Open Graph & SEO basics

1. **ShopFlow Skeleton** — structure before style: Scaffold product listing, product detail, and cart pages with header/nav/main/section/article/footer → no CSS yet, structure only → validate with the W3C HTML validator → confirm the outline in browser dev tools reads like a table of contents
2. **Accessible Checkout & Auth Forms:** Add a proper label for every input on the (stubbed) checkout, login, and register forms → use correct input types (email, tel) → add required/pattern attributes → tab through every form using only the keyboard
3. **ARIA-Enhanced Cart Drawer:** Build the cart drawer as a dropdown/panel → add role, aria-expanded, and aria-live for item-count updates → test with a screen reader or accessibility dev tools → fix every issue the audit flags
4. **Developer Portfolio — semantic scaffold:** Structure your own portfolio (about, projects, contact) with the same semantic rigor → add Open Graph and meta tags so it previews correctly when shared → this becomes the site that showcases ShopFlow later

Study Resources: MDN: HTML Basics · MDN: Learn HTML · web.dev: Learn HTML

2 CSS UPDATED

Layout and visual presentation — box model, modern layout systems, and responsive design, now including component-level responsiveness.

Subtopics: box model & positioning · Flexbox · Grid · media queries · CSS variables & transitions · container queries [NEW] · logical properties [NEW]

1. **ShopFlow Nav + Grid Catalog:** Build the storefront navbar with Flexbox, collapsing to a mobile menu via media query → build the product catalog with CSS Grid using auto-fit/auto-fill → reflow the grid from 4 to 2 to 1 columns and confirm at 3 real viewport widths
2. **Container Query Product Card:** Wrap the existing ProductCard in container-type: inline-size → write breakpoints that switch it from stacked to side-by-side → drop the same card into a narrow sidebar (related items) and the wide main grid → confirm it adapts independently of viewport width
3. **Logical Properties Pass:** Replace left/right-based spacing and positioning across the storefront with margin-inline, padding-block, etc. → flip <html dir="rtl"> and confirm layout mirrors correctly with zero extra CSS

Study Resources: MDN: CSS Reference · CSS-Tricks: A Complete Guide to Flexbox · MDN: CSS Container Queries

3 Tailwind CSS & shadcn/ui UPDATED

Utility-first styling plus the headless-component pattern (Radix primitives + Tailwind) dominating new React libraries — replacing Bootstrap, now a legacy pattern in greenfield React work.

Subtopics: Tailwind utility classes & config · responsive breakpoint prefixes · shadcn/ui: copy-in components, not npm-installed · Radix primitives under the hood · theming with CSS variables

1. **Migrate ShopFlow to Tailwind:** Rebuild the existing storefront's CSS as Tailwind utility classes, no separate stylesheet → use sm:/md:/lg: prefixes for the breakpoints you already defined → delete the old CSS file once parity is confirmed
2. **Design Tokens in Config:** Extend tailwind.config with ShopFlow's brand color palette and font family → apply the tokens across the nav, buttons, and cards → confirm the production build only includes classes actually used
3. **shadcn/ui Component Pull:** Run the shadcn/ui CLI to add Dialog and Select → wire the Dialog into the cart drawer and Select into a sort/filter control → read the generated files — this code now lives in your repo, not node_modules → comment on why "copy the code in" is a different trust model than npm install
4. **Bootstrap-for-Comparison (trimmed):** On a throwaway page only, add Bootstrap via CDN and drop in a navbar + modal using prebuilt classes → note how much JS behavior comes bundled versus what Tailwind+shadcn required you to wire yourself → write 2 bullets on when you'd still choose Bootstrap (e.g. an internal admin tool on a tight deadline)

Study Resources: Tailwind CSS Documentation · shadcn/ui Documentation · Radix UI Primitives

4 JavaScript (ES6+) UPDATED

The language that runs in the browser — modern syntax, the DOM, and async programming, now including the execution model behind the async bugs you'll hit in React.

Subtopics: let/const, arrow functions, destructuring · DOM manipulation & events · Promises & async/await · closures & modules · array methods (map/filter/reduce) · the event loop: microtasks vs. macrotasks [NEW]

1. **ShopFlow Cart in Vanilla JS:** Render the cart from a JS array of items → add/remove/adjust-quantity handlers → compute a running total → persist the cart to localStorage so it survives a refresh
2. **Live Product Search:** Call your real product data (local JSON stand-in for the API) with fetch() as the user types → handle the loading state while waiting → handle the error/empty-results state → render matching products once resolved
3. **Event Loop Prediction Drill:** Add a checkout button whose click handler mixes console.log, setTimeout(fn, 0), and a fetch() call chained with .then() → predict the exact print order on paper first → run it and compare → write one paragraph on why microtasks always drain before the next macrotask

Study Resources: MDN: JavaScript Guide · javascript.info: The Modern JavaScript Tutorial · MDN: Using Promises · MDN: Microtasks and the JS runtime

5 TypeScript

JavaScript with a type system layered on top — catching a whole category of bugs before the code ever runs.

Subtopics: basic types & interfaces · union/intersection types · type narrowing · generics

1. **Type the ShopFlow Domain:** Add Product, CartItem, and Order interfaces matching your Part A Spring Boot DTOs → type every cart function's parameters and return value → fix every compiler error
2. **Discriminated Union: Order Status:** Model order status as a discriminated union (pending | shipped | delivered | cancelled) sharing a status field → write a status-badge function switching on it → confirm TypeScript narrows correctly in each case → add a new status and let the compiler flag the missing case
3. **Strict Mode Migration:** Enable strict: true in tsconfig.json across the ShopFlow codebase → fix every new error, paying special attention to implicit any → confirm the project still builds cleanly

Study Resources: TypeScript Official Handbook · TypeScript: TSConfig Reference · Total TypeScript: Beginner's Tutorial

PHASE 5 SHIP-IT CHECKLIST — TESTING · DEBUGGING · ACCESSIBILITY · PRODUCTION

Testing: HTML validates cleanly (W3C validator); manual cross-browser check in Chrome, Firefox, Safari.
Debugging: comfortable reading a layout with the Elements/Inspector box-model panel and diagnosing a broken Flex/Grid layout without guessing.
Accessibility: every interactive element is keyboard-reachable; landmark regions and heading order are correct; contrast checked with a dev-tools contrast checker.
Production: Tailwind production build purges unused classes; images have explicit width/height to prevent layout shift; portfolio meta tags render correctly in a social-preview debugger.

FINAL PROJECT — SHOPFLOW: STATIC STOREFRONT (REFINED)

- Product listing and detail pages in semantic, accessible HTML
- Fully responsive layout (Grid/Flexbox) styled with Tailwind; at least one component rebuilt to respond to a container query, not just the viewport
- **NEW:** the cart drawer and a filter panel built with shadcn/ui components instead of hand-rolled markup
- Cart in vanilla TypeScript — add/remove, running total, localStorage persistence, fully typed
- Product data from a local JSON file (stand-in for the real API)
- Companion developer portfolio site, live, showcasing this build
- No framework yet — proves you can build a real, accessible UI with just HTML/CSS/TS

This is your frontend fundamentals proof-of-concept, the same way Phase 1's console cart proved your OOP. Phase 6 doesn't rebuild this — it migrates and extends this exact codebase into React, wired to your real Spring Boot API.

PHASE 6 OF 10

Frontend Application Layer

React 19 · Component Architecture & Design Systems · Forms · TanStack Query · State Management · Testing & A11y · Performance · Next.js Primer
Now migrate your Phase 5 storefront into React and connect it to your real Spring Boot backend from Part A. Deployment gets its own full track in Part C — this phase is the application code and the ecosystem tools a real 2026 React codebase actually uses.

Continuity: Topic 1 migrates the Phase 5 static storefront into a React app — it is not rewritten from a blank create-vite template with placeholder content. Every subsequent topic adds one capability (forms, caching, state, tests, performance, SSR) directly onto that same running app.

1 React 19 + TypeScript UPDATED

A component-based UI library, typed with TypeScript, now including React 19's Actions model and the resilience patterns a real app needs beyond the basics.

Subtopics: JSX, props, state, hooks · React 19: Actions, useActionState, useOptimistic, use(), ref-as-prop [NEW] · useReducer for complex local state [NEW] · custom hooks [NEW] · React Router (protected routes) · error boundaries [NEW] · focus management in an SPA [NEW]

1. **Migrate ShopFlow to React:** Scaffold a Vite + React 19 + TypeScript app → port the Phase 5 product listing, detail, and cart markup into typed components, preserving the Tailwind/shadcn styling → confirm visual parity with the static version before adding any new behavior
2. **Auth Pages with React 19 Actions:** Build Login/Register using useActionState and a form Action instead of manual onSubmit + useState plumbing → call your Phase 3 auth API on submit → store the returned JWT → build a ProtectedRoute wrapper redirecting unauthenticated users
3. **Product Listing → Real Data + Custom Hook:** Fetch products from your Spring Boot API, replacing the local JSON → add category filter and search → extract the fetch-and-loading logic into a useProducts custom hook → reuse it on a second page and confirm one shared source of truth
4. **Error Boundary + Focus Management:** Wrap the product routes in an ErrorBoundary with a fallback UI instead of a blank screen → add focus-trapping to the auth modal, returning focus to the trigger button on close → confirm a screen reader announces route changes

Study Resources: React Official Documentation · React 19 Upgrade Guide · React Router Documentation · TypeScript: React Cheatsheet · React: Error Boundaries

2 Component Architecture & Design Systems NEW

How to structure components so the codebase stays maintainable as it grows, and how to formalize the visual language you built in Tailwind into a real design system.

Subtopics: compound components · controlled vs. uncontrolled · prop drilling vs. lifting state up · children-as-function / render props · design tokens as the system's source of truth · when to split a component

1. **Compound Component: Product Detail Tabs:** Build a Tabs API (specs / reviews / shipping) using React Context internally → the parent manages active-tab state while children just render → compare ergonomics to a single component taking a giant props object
2. **Controlled vs. Uncontrolled Audit:** Take two ShopFlow form inputs built as controlled components → rebuild one as uncontrolled using a ref (e.g. the checkout file-upload for a receipt) → write 3 bullets on when you'd actually choose uncontrolled
3. **ShopFlow Design System — Button, Input, Modal, Table, Pagination:** Extract typed Button and Input from your Tailwind/shadcn styles → extract a reusable Modal, Table, and Pagination → promote your Tailwind config tokens (color, spacing, type scale) into a documented token set → replace every ad-hoc styled element across the app with these five components

Study Resources: React: Passing Data Deeply with Context · patterns.dev: Compound Pattern · React: Sharing State Between Components

3 Forms with React Hook Form + Zod NEW

Hand-rolled form state — a useState per field, manual validation — is exactly the repetitive, error-prone work the ecosystem already solved. This is what a real form-heavy PR looks like.

Subtopics: RHF: register, handleSubmit, formState · Zod schema validation · zodResolver integration · field- vs. form-level errors · async validation

1. **Checkout Form — the standard RHF + Zod workflow:** Define a Zod schema for the real checkout fields (name, email, address, card) → wire it to RHF via zodResolver → render field-level errors → confirm submission is blocked until the schema passes
2. **Async Email Availability Check:** On the Register form, check email availability against your Spring Boot API as the user types → debounce the check → show pending/valid/invalid state per keystroke pause
3. **Reusable FormField Component:** Extract a typed FormField combining label + input + error message, using the design-system Input from the previous topic → wire it to RHF's register → use it across checkout and the auth forms so one component renders every form field in the app

Study Resources: React Hook Form Documentation · Zod Documentation · React Hook Form: Zod Resolver

4 Axios + TanStack Query UPDATED

Axios stays for the client/interceptor layer; the fetching and caching pattern is TanStack Query, not manual useEffect. Caching, background refetching, and deduplication are the actual hard parts of data fetching.

Subtopics: Axios centralized instance & interceptors · TanStack Query: queries & mutations [NEW] · cache invalidation & refetching [NEW] · optimistic updates [NEW] · JWT attach + auto-refresh on 401

1. **Axios API Client Setup:** Create one axios.create() instance with baseURL and default headers → request interceptor attaches the JWT automatically → response interceptor handles 401 with auto token refresh
2. **Migrate Product Fetching to TanStack Query:** Replace the useProducts hook's manual useEffect fetch with useQuery, passing the Axios instance as fetcher → delete the manual loading/error state → confirm a second visit shows cached data instantly while revalidating in the background
3. **Optimistic Add-to-Cart Mutation:** Wire "add to cart" through useMutation → update the cart UI optimistically before the server responds, using React 19's useOptimistic where it fits → configure onError to roll back → throttle the network in dev tools and watch the optimistic state, then the real confirmation

Study Resources: Axios Documentation · Axios: Interceptors Guide · TanStack Query Documentation · TanStack Query: Optimistic Updates

5 State Management: Context vs. Zustand NEW

Separating server state (TanStack Query's job) from client/UI state (Context or Zustand's job) is the distinction most junior React work gets wrong.

Subtopics: server state vs. client state · Context: when it's the right size · Context's re-render cost at scale · Zustand: a minimal external store · choosing per-feature, not project-wide

1. **Cart State in Context:** Move the cart's quantity controls, remove button, and running total into a Context provider → persist across a page refresh
2. **Reproduce Context's Re-render Problem, Then Fix:** Add a fast-changing value (live search-input state) into the existing cart Context → watch every consumer re-render on every keystroke via React DevTools' render highlighting → move just that value into a Zustand store → confirm unrelated components stop re-rendering
3. **Server vs. Client State Audit:** List every piece of state in ShopFlow so far → mark each "server state" (TanStack Query) or "client state" (Context/Zustand/useState) → move any misplaced state to the right layer → write 2 bullets explaining the distinction to a teammate

Study Resources: React: Passing Data Deeply with Context · Zustand Documentation · TkDodo's blog: Practical React Query (server vs. client state)

6 Types of APIs You'll Work With

Recognizing API styles beyond the REST/GraphQL you've already built.

Subtopics: REST (recap) · GraphQL (recap) · WebSocket/Server-Sent Events · Webhooks · gRPC & SOAP (awareness only)

1. **Live Order Status Widget:** Connect to a WebSocket/SSE endpoint from React → subscribe to order-status events → update the ShopFlow Order History page in real time → confirm no polling requests are happening
2. **Compare Polling vs. WebSocket:** Build the same live-status feature with setInterval polling first, then with WebSocket/SSE → compare network-tab traffic on both → write 3 bullets on the trade-offs

Study Resources: MDN: WebSockets API · MDN: Server-Sent Events · Spring: WebSocket Support Reference

7 Testing & Accessibility Audits NEW

A frontend that ships zero tests, and zero automated accessibility checks, trains habits that don't survive a real team's CI pipeline. This lands right after components are built.

Subtopics: Vitest basics · React Testing Library: query by role, not implementation detail · mocking API calls · axe-core in component tests (vitest-axe) [NEW] · one Playwright E2E flow · @axe-core/playwright automated a11y scans [NEW] · what to test vs. what not to

1. **Component Tests: Design System:** Write Vitest + RTL tests for Button, Input, and Modal → query by role/label, not class name or test-id → test disabled state and click handlers → add a vitest-axe assertion to each so a regression that breaks accessible naming fails CI, not just a human reviewer
2. **Mocked API Component Test:** Test the Product Listing page with the Axios/TanStack Query call mocked → assert the loading skeleton renders first, then the product list → assert the empty-results message renders when the mock returns []
3. **Playwright E2E + Automated A11y Scan:** Write one Playwright test: log in → add a product to cart → complete checkout → assert the order appears in Order History → run @axe-core/playwright against the checkout and product-detail pages in the same test → fail the test on any serious/critical violation → run headed once, then headless in CI mode

Study Resources: Vitest Documentation · React Testing Library Documentation · Playwright Documentation · @axe-core/playwright · Kent C. Dodds: Common Testing Mistakes

8 Performance: Code-Splitting, Suspense & Core Web Vitals NEW

The same "measure before optimizing" instinct already used well in Part A's database-indexing phase, applied to the frontend — profile first, then fix the specific thing that's actually slow.

Subtopics: React.lazy & Suspense · route-based code-splitting · bundle-size awareness · Lighthouse & Core Web Vitals (LCP, CLS, INP) · when useMemo/useCallback actually help

1. **Baseline Lighthouse Pass:** Run a Lighthouse audit on ShopFlow as it stands → record LCP, CLS, and INP → identify the single largest JS bundle chunk in the network tab
2. **Code-Split Order History with Suspense:** Wrap the Order History route in React.lazy + Suspense, with a skeleton fallback → confirm its JS chunk loads only when visited → rerun Lighthouse and compare initial bundle size before/after
3. **Prove Memoization Before You Reach For It:** Find a component re-rendering unnecessarily using the React DevTools Profiler → add useMemo/useCallback only where the profiler shows a real cost → remove one "just in case" memoization and confirm no measurable difference → write one paragraph on why memoizing everything by default is itself an anti-pattern

Study Resources: React: React.lazy Reference · React: Suspense · web.dev: Core Web Vitals · Chrome DevTools: Lighthouse · React DevTools Profiler

9 Frontend Dev Tools UPDATED

The tools you'll reach for constantly while building the above — reference, not a project list.

Subtopics: npm/pnpm/yarn · Vite: dev server, HMR, build config · ESLint + Prettier · browser DevTools (Network, Console, Application) · React Developer Tools · Postman/Insomnia · Storybook for isolated component development [NEW]

1. **Dev Tools Fluency Pass:** Inspect a failed ShopFlow API call in the Network tab and identify the exact response body → use the Application tab to inspect the stored JWT and cart persistence → profile a re-render with React DevTools
2. **Storybook for the Design System:** Add Storybook to the project → write stories for Button and Modal covering default/disabled/loading states → confirm you can develop and visually check a component without running the full app

Study Resources: Vite Documentation · ESLint Documentation · Chrome DevTools Documentation · Storybook Documentation

10 Next.js Primer NEW

Enough exposure to SSR, file-based routing, and Server Components to see the alternative to the SPA-only model — one guided page rebuild, not a full migration.

Subtopics: file-based routing · Server vs. Client Components · SSR/SSG basics · structured data (JSON-LD) for product SEO · why this solves SPA SEO · when you'd reach for Next.js vs. stay SPA

1. **Rebuild Product Listing in Next.js:** Create a fresh Next.js app → rebuild just the Product Listing page using the App Router, fetching in a Server Component instead of useQuery → compare: no loading skeleton needed, because the HTML arrives with data already in it
2. **Client Component Boundary:** Add an "add to cart" button needing interactivity → mark it "use client" → confirm the rest of the page stays server-rendered → write 3 bullets on how you'd decide the client/server boundary on a real page
3. **SEO Comparison + Structured Data:** View-source the original SPA's Product Listing and note the empty shell → view-source the Next.js rebuild and note the fully-rendered HTML → add JSON-LD product structured data to the page → explain in a comment why this matters for a crawler that doesn't execute JavaScript

Study Resources: Next.js Documentation · Next.js: App Router · React: Server Components

PHASE 6 SHIP-IT CHECKLIST — TESTING · DEBUGGING · ACCESSIBILITY · PRODUCTION

Testing: Vitest/RTL suite green in CI; one Playwright E2E flow covering login → cart → checkout; mocked-network tests for the data layer.
Debugging: comfortable reading a React DevTools Profiler flame chart and an Axios interceptor stack trace to find where a 401 loop or a stale-cache bug actually originates.
Accessibility: automated axe-core scans pass with zero serious/critical violations on checkout, cart, and auth flows; focus management verified with a screen reader, not just visually.
Production: Lighthouse Core Web Vitals recorded before/after code-splitting; JWT refresh and error boundaries verified under throttled/offline network conditions; env-based config for API base URL.

FINAL PROJECT — SHOPFLOW: REACT FRONTEND, PRODUCTION-READY

- The Phase 5 storefront, migrated (not rebuilt) into React 19 + TypeScript, wired to the real Spring Boot API from Part A
- Centralized Axios client with interceptors for JWT attach + auto-refresh on 401
- Product and cart data fetched via TanStack Query instead of manual useEffect, with an optimistic add-to-cart mutation
- Checkout and auth forms built with React Hook Form + Zod
- Server state (TanStack Query) and client state (Context/Zustand) kept deliberately separate
- Live order status via WebSocket/SSE instead of polling
- Design-system component library (Button, Input, Modal, Table, Pagination) used consistently, developed and documented in Storybook
- Full auth flow: register, login, protected routes, logout
- Vitest + RTL component test suite with axe-core assertions, plus one Playwright E2E flow that includes an automated accessibility scan
- At least one route code-split with React.lazy + Suspense, with a before/after Lighthouse Core Web Vitals comparison
- One page rebuilt in the Next.js primer with structured data and a view-source SEO comparison against the SPA version
- Companion developer portfolio site updated with a ShopFlow case study, screenshots, and a link to the live app

Deploying this frontend (and the Spring Boot backend behind it) to a live URL is covered in full in Part C — Deployment & Cloud, right after External APIs.

Part B — Progress Tracker

Phase 5 — Frontend Foundations

- ☐ HTML
- ☐ CSS UPDATED
- ☐ Tailwind CSS & shadcn/ui UPDATED
- ☐ JavaScript (ES6+) UPDATED
- ☐ TypeScript
- ☐ Phase 5 Ship-It Checklist
- ☐ Final Project: Static Storefront

Phase 6 — Frontend Application Layer

- ☐ React 19 + TypeScript UPDATED
- ☐ Component Architecture & Design Systems NEW
- ☐ Forms: React Hook Form + Zod NEW
- ☐ Axios + TanStack Query UPDATED
- ☐ State Management: Context vs. Zustand NEW
- ☐ Types of APIs You'll Work With
- ☐ Testing & Accessibility Audits NEW
- ☐ Performance & Core Web Vitals NEW
- ☐ Frontend Dev Tools UPDATED
- ☐ Next.js Primer NEW
- ☐ Phase 6 Ship-It Checklist
- ☐ Final Project: React Frontend, Production-Ready

SHOPFLOW CURRICULUM · PART B AUDIT · 2026–2027 EDITION

Frontend Engineering Curriculum Audit

A professional audit of Phases 5–6 (Frontend Foundations & Application Layer) against 2026–2027 hiring bars at product-engineering companies — Vercel, Meta, Shopify, Stripe, Microsoft, and Atlassian.

Auditor Role: Staff Frontend Engineer / Hiring Manager | Method: Topic-by-Topic Verdict | Constraint: 8–12 hrs/week | Continuity: Single ShopFlow Codebase

SCOPE
15 existing topics across Phase 5 (Foundations) & Phase 6 (Application Layer), plus a gap analysis against 50+ candidate 2026 topics.

VERDICT SCALE
KEEP · UPDATE · EXPAND · REORDER · SPLIT · REMOVE · ADD NEW

BAR TEST
Every recommendation must improve real-world skill, reflect current industry practice, or reduce hiring/maintainability risk — never "newer for its own sake."

NET TIME IMPACT
+16.5 hrs required (§4) · +7 hrs judgment/practice (§5) · +14 hrs optional (§6, not counted)

Headline finding: this is an unusually mature roadmap. Of the ~50 topics commonly cited as 2026 frontend gaps, this edition already ships 30+ of them (container queries, RSC, TanStack Query, RHF+Zod, Zustand, Vitest/RTL/axe-core/Playwright, Core Web Vitals, Storybook, design tokens, WebSocket/SSE, Next.js primer). The audit below keeps that structure intact and targets the handful of items that are genuine, high-leverage gaps rather than padding the syllabus.

1 · Executive Score Summary (Post-Audit, §4 + §5 Implemented)

10/10	10/10	10/10	9.5/10
OVERALL	HIRING READINESS	PORTFOLIO STRENGTH	MODERNITY
With §4 (technical) and §5 (judgment) additions in place	Junior–early-mid bar, plus interview-ready reasoning	One evolving app, plus a visible ADR log proving judgment	Held just under 10 on principle — see §9

2 · Phase 5 Audit — Frontend Foundations

Topic	Status	Reason	Industry Justification
HTML	KEEP	Structure-before-style sequencing and the ARIA cart-drawer exercise already teach markup as an engineering decision, not decoration.	Real PRs get blocked in review for div-soup and missing landmarks; this is exactly what a Shopify/Stripe frontend review checklist flags first.
CSS	EXPAND	Container queries and logical properties are the right modern core. Add the browser rendering pipeline (Style → Layout → Paint → Composite) as a subtopic, read directly off the DevTools Performance panel against the existing Grid/Flexbox exercise.	Every senior CSS debugging conversation ("why is this reflowing," "why is this layer promoted") assumes this model. It's the single most common gap in candidates who "know CSS" but can't explain a jank bug.
Tailwind CSS & shadcn/ui	KEEP	Correctly frames shadcn/ui's copy-in model as a different trust boundary than npm install, and keeps a trimmed Bootstrap comparison for legacy-maintenance literacy rather than nostalgia.	Radix-primitive, copy-in components are the dominant pattern in new React component libraries as of 2026; Bootstrap survives only in legacy/internal-tools contexts, which the curriculum already frames correctly.
JavaScript (ES6+)	EXPAND	The event-loop prediction drill is excellent. Add one short subtopic on closures & cleanup as a memory-leak source (a detached DOM listener, an uncleared interval) — a natural pairing with the existing closures subtopic.	"My tab's memory keeps climbing" is one of the most common real debugging tickets a junior engineer is handed in month one; the mechanism is closures, which this phase already teaches.
TypeScript	KEEP	Discriminated unions on order status plus a strict-mode migration is the correct depth for this stage — narrow, applied, not academic.	Strict-mode literacy and discriminated unions are what separates "TypeScript as a linter" from "TypeScript as a design tool," which is exactly what a code review at this level checks for.

3 · Phase 6 Audit — Frontend Application Layer

Topic	Status	Reason	Industry Justification
React 19 + TypeScript	EXPAND	Actions, useOptimistic, error boundaries and focus management are the right 2026 surface. Attach a small error-monitoring subtopic (wire Sentry into the existing ErrorBoundary) rather than opening a new topic.	Shipping an ErrorBoundary with no telemetry behind it is a common junior tell in interviews — "how would you know this fired in production?" is a near-guaranteed follow-up question.
Component Architecture & Design Systems	KEEP	Compound components, controlled/uncontrolled audits, and promoting Tailwind tokens into a documented system is precisely how real design-system work is scoped.	This is the exact skill gap between "uses a component library" and "can build one," which is the differentiator at design-system-heavy shops like Shopify and Atlassian.
Forms (RHF + Zod)	KEEP	Correctly treats hand-rolled form state as solved-problem territory and goes straight to schema validation and async field checks.	RHF + Zod (or an equivalent schema-first pairing) is close to a default in 2026 React codebases; interviewers assume familiarity with the pattern even if not the exact library.
Axios + TanStack Query	KEEP	Correctly separates the interceptor/client layer (Axios) from caching/refetching (TanStack Query) instead of treating them as competitors.	Manual useEffect data fetching is now treated as a code-smell in review at most product companies; this is the right replacement to teach as default.
State Mgmt: Context vs. Zustand	KEEP	The re-render reproduction exercise (prove the problem, then fix it) is the correct pedagogy — most courses assert the trade-off instead of demonstrating it.	"Server state vs. client state" is one of the most common system-design-style questions in mid-level React interviews; this topic answers it experientially.
Types of APIs	KEEP	WebSocket/SSE with a real polling-vs-push comparison already covers the real-time gap. gRPC/SOAP are appropriately scoped as awareness-only.	Most frontend roles never write gRPC by hand but do need to recognize it in a service diagram; awareness-only is the correct depth, not a hands-on project.
Testing &	KEEP	Vitest/RTL + axe-core assertions baked into component tests, plus one	Automated axe-core in CI (not just manual audits) is what

Accessibility Audits		Playwright E2E with an automated a11y scan, is genuinely CI-grade practice for this level.	separates “we care about a11y” from “we enforce a11y,” which is increasingly a stated bar at Microsoft and Shopify.
Performance & Core Web Vitals	EXPAND	Measure-first is correctly emphasized, but the topic promises a “scored Lighthouse CI gate” in the changelog without actually building one. Add a real Lighthouse CI config, a bundle-analyzer pass, and responsive-image/font-loading practice.	A candidate who can read a bundle-analyzer treemap and explain an LCP regression from an unsized hero image is doing exactly what a performance-focused PR review looks like at Vercel or Shopify.
Frontend Dev Tools	KEEP	Storybook addition is well-placed right after the design-system topic, where it has something real to document.	Storybook remains the standard for isolated component development and design-system documentation in 2026 React shops.
Next.js Primer	EXPAND	Correctly scoped as a primer, not a migration. Add one short subtopic explicitly naming hydration (why the Client Component still needs to “wake up” in the browser) since the SEO comparison exercise surfaces it implicitly but never names it.	“Explain hydration” is a near-universal SSR interview question; the primer already builds the exact page that demonstrates it, it just needs to be named and explained.

4 · Recommended Additions (Required — Fit Inside Existing Phases)

Each of these is scoped as a subtopic or micro-topic riding on an exercise that already exists, not a new standalone project — this is how the +16.5 hrs stays inside a one-week pace extension rather than a syllabus rewrite.

Frontend Security Fundamentals — CSP, XSS, Clickjacking, OAuth & JWT-Storage Trade-offs ADD NEW

Reason: ShopFlow already stores a JWT and renders user-supplied content (reviews, product descriptions) with zero explicit security topic. This is the sharpest real gap in the curriculum: the app is exercising exactly the surfaces (auth tokens, rendered user input) that create XSS and token-theft risk, without ever naming the risk.

Industry justification: A security review at any product company will ask why a JWT is in localStorage vs. an httpOnly cookie, and whether user-generated content is sanitized before render. Teaching this against ShopFlow’s real auth flow — not a toy example — is exactly how staff engineers actually onboard juniors to security thinking.

Difficulty: Medium **Est. time:** 4–5 hrs **Placement:** Phase 6, placed directly after Axios + TanStack Query

Browser Rendering Pipeline & Critical Rendering Path ADD NEW

Reason: Style → Layout → Paint → Composite is the mental model behind every “why is this janky” question the JS event-loop drill already trains students to ask about async code — CSS deserves the equivalent for rendering.

Industry justification: Directly strengthens browser fundamentals and debugging ability — two of the audit’s core success criteria — and is a two-minute DevTools Performance-panel exercise, not new tooling.

Difficulty: Low **Est. time:** 1.5 hrs **Placement:** Subtopic of Phase 5 CSS

Motion & Micro-interactions (Framer Motion / View Transitions API) ADD NEW

Reason: ShopFlow already has a cart drawer and route transitions with no motion layer; animating the existing drawer open/close and the checkout step transition is a natural, low-cost extension rather than a new surface.

Industry justification: Motion is now a visible differentiator in portfolio review at product-design-forward companies (Vercel, Stripe); the View Transitions API specifically is the 2026-native browser answer worth knowing before reaching for a library.

Difficulty: Low–Medium **Est. time:** 3.5 hrs **Placement:** Subtopic of Component Architecture & Design Systems

Feature Flags (Conditional Checkout Flow) ADD NEW

Reason: Gate a second checkout-step order (e.g., address-before-payment vs. payment-before-address) behind a boolean flag read from Context/Zustand — a two-hour exercise that teaches the pattern without pulling in a vendor SDK.

Industry justification: Feature flags are close to universal at companies shipping continuously; understanding that a flag is just conditionally-rendered state (not magic) strengthens architecture understanding, one of the audit’s named criteria.

Difficulty: Low **Est. time:** 2 hrs **Placement:** Micro-topic inside State Management

Bundle Analysis, Lighthouse CI Gate & Responsive Images/Fonts EXPAND

Reason: Add a bundle-analyzer pass on the existing Vite build, a real Lighthouse CI config that fails the build under a score threshold, and a responsive-image/font pass (srcset/sizes, next/image in the primer, font-display) on assets ShopFlow already has. Delivers on the topic’s own changelog promise of a “scored Lighthouse CI gate.”

Industry justification: A scored, CI-enforced performance budget — not just a one-time audit — is what “we take performance seriously” means operationally at Vercel/Shopify-tier teams; this closes the gap between the changelog’s stated intent and the actual exercises.

Difficulty: Medium **Est. time:** 3.5 hrs **Placement:** Expands existing Performance topic

Memory Leaks & useEffect Cleanup ADD

Reason: Pair the vanilla-JS closure gotcha with a matching React example — an uncancelled fetch or unremoved event listener inside a live-search or WebSocket subscription ShopFlow already has.

Industry justification: Directly strengthens debugging ability; a rising memory graph in the Chrome Memory tab is one of the most common “explain what’s happening” prompts in senior-shadowed junior interviews.

Difficulty: Low **Est. time:** 2 hrs **Placement:** Subtopic of Phase 5 JavaScript, revisited in Phase 6 React

5 · Path to a Perfect 10

Everything through §4 closes technical gaps. What’s left is not more technical depth — testing, accessibility, performance, and security are already CI-grade — it’s evidence of engineering judgment: can the candidate explain why they made a decision, work inside a review process, and reason about a system out loud under interview conditions. These three additions are what a staff-level reviewer is actually still missing at 9.2, and they ride on exercises ShopFlow already has.

Architecture Decision Records (ADRs) — Formalize the "Write 2–3 Bullets" Habit ADD NEW

Reason: The curriculum already asks for reflection ("write 2 bullets on when you'd choose uncontrolled," "write 3 bullets on the trade-offs") at eight separate points — but those bullets currently live nowhere durable. Turn each into a committed docs/adr/000N-title.md file (context / decision / consequences) so the reasoning becomes a visible, versioned part of the repo instead of a throwaway thought.

Industry justification: ADRs are how real engineering teams make decisions legible to reviewers and future maintainers. A portfolio repo with 8+ real ADRs — Tailwind over Bootstrap, TanStack Query over manual fetch, Zustand over Context-everywhere — is a concrete artifact a hiring panel can read in two minutes and immediately trust the candidate's judgment, not just their code.

Difficulty: Low **Est. time:** 1.5 hrs setup + ongoing **Placement:** Runs through both phases

Code Review & Collaboration — PR Descriptions, Self-Review, Review Checklist ADD NEW

Reason: Nothing in the roadmap currently teaches how ShopFlow's work gets reviewed. Pick one real feature (the optimistic add-to-cart mutation) and write it up as a proper PR: a description explaining the "why," a self-review against a short checklist (tests included? a11y checked? bundle impact considered?), and — if the learner has any developer contacts — one real external review.

Industry justification: Review literacy is assessed as heavily as raw coding ability in real hiring loops, and a self-taught learner's biggest blind spot is never having been on either side of a PR. This is the single most common gap between "can build ShopFlow" and "would survive a first sprint on a real team."

Difficulty: Low **Est. time:** 2.5 hrs **Placement:** Short topic, end of Phase 6, before the final project

Frontend System Design Capstone & Mock Interview ADD NEW

Reason: Given a prompt ("ShopFlow just got 10x traffic — walk me through what breaks first and what you'd change"), produce a timed, one-page write-up covering rendering strategy, caching, code-splitting, and CDN placement — then say it out loud in 5 minutes, as if presenting to a panel. Every fact needed is already something the learner built in Phases 5–6; this is purely about synthesizing and articulating it.

Industry justification: Staff/senior interviewers at Vercel, Shopify, and Stripe-tier companies weight verbal system-reasoning as much as the take-home code. A candidate who's rehearsed explaining ShopFlow's architecture under time pressure walks into a real interview measurably more confident than one who's only ever written the code silently.

Difficulty: Medium **Est. time:** 3 hrs **Placement:** Phase 6 capstone

Why this closes the gap to 10/10: Hiring Readiness and Portfolio Strength were the two axes still shy of perfect — both measure how a reviewer perceives the candidate, not just what the code does. ADRs make judgment visible, the PR exercise proves collaboration literacy, and the capstone proves the candidate can defend their own architecture live. Combined with the already-CI-grade technical bar from §2–§4, that's the full picture a "10/10, no notes" review is actually checking for.

6 · Optional Enrichment Track (Not Counted Against the 8–12 hr/week Pace)

These are real 2026 skills but fail the "necessary for this stage" test at the given pace — either niche to specific roles, better suited to Part C (Deployment & Cloud), or high-cost relative to payoff for a generalist junior track. Offered as an appendix for learners with slack time, never as a blocker to finishing Part B.

Topic	Status	Bucket	Why It's Optional, Not Removed
PWA / Service Workers / Offline-first / IndexedDB	ADD	Optional	Genuinely valuable, but deployment-adjacent (caching strategy, install prompts) and better sequenced alongside Part C's hosting/CDN topics than bolted onto an already-dense Phase 6.
Data Visualization (Recharts, not raw D3)	ADD	Optional	A lightweight ShopFlow "Sales Dashboard" admin page pays off for candidates targeting data-heavy product companies (Stripe), but isn't universal enough to require of every learner.
Internationalization / Localization / RTL libraries	ADD	Optional	The CSS phase already exercises the RTL mechanics via logical properties; a full i18n library (next-intl) is a real but non-blocking extension.
Visual Regression Testing (Chromatic/Percy)	ADD	Optional	Natural extension of the existing Storybook work, but adds a paid/hosted tool dependency that shouldn't gate a self-study track.
Micro Frontends / Module Federation (overview)	ADD	Optional	Awareness-only reading, appended to Next.js Primer's study resources — correctly out of scope as a hands-on project at this level, per the original request.
Monorepo / Turborepo / pnpm workspaces / npm publishing	DEFER → Part C	Restructuring	Restructuring ShopFlow's frontend + Part A's Spring Boot backend into a monorepo is a deployment/repo-architecture decision — it belongs with Part C, not bolted onto Part B.

7 · Unnecessary / Not-Needed Concepts

No existing Part B content should be removed. The roadmap is already lean — every current topic clears the "improves real-world skill" bar. The items below are candidate additions from the original gap list that this audit explicitly declines, to protect the 8–12 hr/week pace.

- **Raw D3.js** — overkill for product UI work; Recharts (optional, §6) covers the 90% case with a fraction of the learning curve.
- **CSS-in-JS as a taught pattern** — a declining approach in new 2026 codebases now that Tailwind + shadcn/ui is the dominant pairing; one comparison paragraph is enough, not a project.
- **CSS Modules as a taught pattern** — legacy-maintenance awareness only; not worth hands-on time alongside Tailwind.
- **gRPC / SOAP hands-on implementation** — correctly already awareness-only in "Types of APIs"; a junior frontend role essentially never hand-writes these.
- **Web Workers / Shared Workers as a project** — real but rare at this level; one paragraph of awareness is proportionate, a full exercise is not.
- **Canvas as a standalone topic** — SVG (already implicit via icon/shadcn work) and Recharts cover the practical need; raw Canvas is niche outside games/creative-coding roles.
- **Full Storybook visual-testing pipeline** — Storybook itself stays (already added); the paid visual-regression layer on top is optional (§6), not required.

8 · Final Revised Curriculum Structure

Phase 5 — Frontend Foundations (unchanged scope, +1 subtopic)

- ☐ HTML
- ☐ CSS EXPANDED — adds Browser Rendering Pipeline & Critical Rendering Path
- ☐ Tailwind CSS & shadcn/ui
- ☐ JavaScript (ES6+) EXPANDED — adds Closures & Memory Leaks
- ☐ TypeScript

Phase 6 — Frontend Application Layer (+2 new topics, 3 expanded)

- ☐ React 19 + TypeScript EXPANDED — adds Error Monitoring (Sentry)
- ☐ Component Architecture & Design Systems EXPANDED — adds Motion & Micro-interactions
- ☐ Forms: React Hook Form + Zod
- ☐ Axios + TanStack Query

- ☐ Phase 5 Ship-It Checklist
- ☐ Final Project: Static Storefront

- ☐ Frontend Security Fundamentals NEW
- ☐ State Management: Context vs. Zustand EXPANDED — adds Feature Flags micro-topic
- ☐ Types of APIs You'll Work With
- ☐ Testing & Accessibility Audits
- ☐ Performance & Core Web Vitals EXPANDED — adds Bundle Analysis, Lighthouse CI gate, responsive images/fonts
- ☐ Frontend Dev Tools
- ☐ Next.js Primer EXPANDED — adds Hydration & Streaming SSR (awareness)
- ☐ Code Review & Collaboration NEW
- ☐ Frontend System Design Capstone & Mock Interview NEW
- ☐ Phase 6 Ship-It Checklist
- ☐ Final Project: React Frontend, Production-Ready — now includes an ADR log

Optional Enrichment Appendix (self-paced, not on the critical path): PWA & Offline Basics (Service Worker, IndexedDB) · Data Visualization with Recharts (Admin Dashboard page) · Internationalization Pass (next-intl) · Visual Regression Testing (Chromatic/Percy) · Micro Frontends & Module Federation (reading only)

Revised pace: 10–12 weeks at 8–12 hrs/week (was 8–10 weeks pre-audit). The two-week extension absorbs Security, the Performance expansion, and the §5 Path-to-10 additions — the items with real new surface area — while every other addition rides on an exercise ShopFlow already has.

9 · Final Assessment

10/10	10/10	10/10	9.5/10
OVERALL SCORE	HIRING READINESS	PORTFOLIO STRENGTH	MODERNITY SCORE

Scores reflect the roadmap with §5's Path-to-10 additions implemented. Modernity is intentionally held just under a perfect score — chasing the newest possible stack on every axis (e.g. mandating React Compiler or a bleeding-edge meta-framework) would violate the audit's own rule against recommending change for novelty's sake, and 9.5 already reflects a fully current, defensible 2026 stack.

Category	Summary
5. Missing Concepts (Addressed Above)	Security fundamentals, critical rendering path, motion/micro-interactions, feature flags, memory-leak debugging, a real Lighthouse CI gate (§4) — plus ADRs, code review literacy, and a system-design capstone (§5).
6. Unnecessary Concepts	Raw D3, CSS-in-JS/CSS Modules as taught patterns, gRPC/SOAP hands-on, Web Worker projects, standalone Canvas — see §7.
7. Recommended Additions	6 required (§4, +16.5 hrs), 3 professional-practice additions (§5, +7 hrs), and 5 optional (§6, +14 hrs, self-paced, not gating).
8. Recommended Removals	None. Every existing exercise clears the audit bar; the roadmap was already lean going in.
9. Final Revised Curriculum Structure	See §8 — same two-phase, ShopFlow-continuous structure, with 4 new topics, 5 expanded topics, a running ADR practice, and an optional enrichment appendix that never blocks the critical path.

10 · Is This Roadmap Competitive With Junior Hires at Shopify, Vercel, Stripe, Atlassian, and Microsoft?

Yes — with §4 and §5 implemented, this clears a 10/10 bar. The pre-audit curriculum already covered the areas that most self-study roadmaps skip — automated accessibility in CI, TanStack Query's cache model, a real design-system extraction, and a Next.js primer scoped honestly as a primer. §4 closed the remaining technical gaps: security literacy, rendering-pipeline fluency, and a CI-enforced performance budget. §5 closed the judgment gap that technical skill alone can't: a committed ADR log makes the candidate's reasoning visible, the code-review topic proves they can operate inside a real team's process, and the system-design capstone proves they can defend their own architecture live, under interview conditions. Combined with one continuously evolving ShopFlow codebase as the portfolio artifact, a learner finishing this revised Part B doesn't just meet the junior bar at these companies — they walk in with the artifacts and rehearsed reasoning a panel is actually screening for.